

Práctica 1

Mapeado 3D de Interiores y Sigue Líneas

**Alba Carmona Quirante
Marina Navarro Soler**

Contents

1	Enlace	2
2	Introducción	2
3	Propuesta	3
4	Experimentación	4
4.1	Extracción de características	4
4.1.1	Detectores	4
4.1.2	Filtrado de descriptores inválidos	5
4.1.3	Conversión de nubes RGB a XYZ	5
4.2	Encontrar emparejamientos	7
4.3	Algoritmo RANSAC	7
4.4	Construir el mapa	7
4.5	Uso de ICP	7
4.6	Uso de TF	8
4.7	Arquitectura del nodo ROS2	8
4.7.1	Selección dinámica del pipeline	9
4.7.2	Reducción de frecuencia	9
4.8	Comparación	9
4.9	Problemas encontrados durante la práctica	9
5	Conclusiones	11

1 Enlace

Enlace al vídeo del recorrido de la casa resultante:

https://drive.google.com/file/d/1GEPqGniI0gpOhC_WWTMeQiIAIvbtAbtd/view

2 Introducción

El mapeado tridimensional del entorno es un problema fundamental en el ámbito de la robótica móvil y la percepción artificial. Disponer de una representación 3D del entorno permite a los robots comprender mejor el espacio que les rodea, localizar objetos, planificar trayectorias o navegar de forma autónoma en entornos desconocidos. Para construir este tipo de representaciones es habitual utilizar sensores de profundidad capaces de capturar información geométrica del entorno.

Uno de los enfoques más utilizados para generar estos modelos consiste en trabajar con nubes de puntos [1], que representan el entorno mediante un conjunto de puntos en el espacio tridimensional. Cada captura del sensor proporciona una visión parcial del entorno, por lo que es necesario combinar múltiples observaciones para poder reconstruir una representación más completa. Este proceso se conoce como registro de nubes de puntos, y consiste en calcular la transformación que permite alinear diferentes capturas dentro de un mismo sistema de referencia.

Para abordar este problema existen distintas técnicas basadas en la detección de características, el cálculo de descriptores y la estimación de transformaciones entre observaciones. Estas técnicas permiten identificar regiones similares entre distintas capturas y estimar la rotación y traslación que mejor alinean la información obtenida.

3 Propuesta

Para resolver el problema del registro de nubes de puntos se ha desarrollado un nodo en ROS2 encargado de procesar las nubes capturadas por la cámara de profundidad del robot y construir un mapa 3D del entorno. El sistema sigue un enfoque incremental, en el que cada nueva nube se registra con la anterior para estimar la transformación entre ambas y posteriormente integrarla en un mapa global.

En primer lugar, las nubes de puntos se filtran utilizando VoxelGrid para reducir el número de puntos y disminuir el coste computacional. A continuación, se realiza la extracción de características, detectando keypoints y calculando descriptores que permitan comparar distintas nubes. También vamos a utilizar diferentes combinaciones de detectores y descriptores, como Harris con SHOT y ISS con FPFH.

Posteriormente se buscan correspondencias entre las características de las nubes consecutivas. A partir de estas correspondencias se estima la transformación entre nubes utilizando el algoritmo RANSAC [2], que permite eliminar emparejamientos incorrectos y obtener una transformación más robusta.

Finalmente, la nube actual se transforma al sistema de referencia global y se añade al mapa acumulado. Para evitar que el número de puntos crezca demasiado, se vuelve a aplicar el filtro VoxelGrid [3], manteniendo así un mapa más compacto y manejable.

4 Experimentación

4.1 Extracción de características

En cuanto a la extracción de características, se han evaluado diferentes combinaciones de detectores y descriptores, como Harris + SHOT e ISS + FPFH. La elección de estas combinaciones se basa en el compromiso entre robustez y coste computacional. Mientras que Harris detecta un mayor número de keypoints, ISS proporciona puntos más estables y discriminativos. De forma similar, SHOT ofrece descriptores más ricos pero con mayor coste, mientras que FPFH resulta más eficiente, lo que lo hace más adecuado para aplicaciones en tiempo real.

En esta fase se implementa el primer paso necesario para poder registrar y alinear las nubes de puntos capturadas por el robot. Cada vez que el robot se mueve, el sensor Kinect genera una nube de puntos que representa el entorno en tres dimensiones. Estas nubes pueden contener decenas de miles de puntos, por lo que trabajar directamente con todos ellos resulta costoso computacionalmente. El objetivo de esta fase es reducir la información de la nube a un conjunto de puntos relevantes (keypoints [4]) y describir la forma del entorno alrededor de cada uno de ellos.

Antes de aplicar los algoritmos de detección de características, se realiza un preprocesado de la nube utilizando el filtro VoxelGrid. Este filtro divide el espacio en pequeñas celdas tridimensionales llamadas voxels y sustituye todos los puntos dentro de cada celda por un único punto representativo.

De esta manera se obtiene una versión simplificada de la nube que mantiene la estructura general del entorno, pero con un número menor de puntos. Esto permite reducir el tiempo de cálculo en las etapas siguientes.

Se han implementado múltiples combinaciones de detectores y descriptores, permitiendo seleccionar dinámicamente el pipeline de procesamiento. En concreto, se han añadido las siguientes configuraciones:

- Harris + SHOT [5] [6]
- Harris + FPFH [7]
- ISS + SHOT [8]
- ISS + FPFH

Esto permite analizar el comportamiento de distintos métodos de extracción de características bajo las mismas condiciones.

4.1.1 Detectores

En esta etapa se emplean distintos detectores de puntos clave con el objetivo de identificar las zonas más representativas de la nube de puntos.

El detector Harris 3D identifica puntos donde existen cambios significativos en la geometría local, como esquinas o bordes. Para ello analiza la variación de la superficie en un vecindario alrededor de cada punto y asigna una medida de respuesta. Los puntos con mayor respuesta se consideran más característicos. Este método tiende a detectar un mayor número de keypoints, lo que puede ser beneficioso para encontrar correspondencias, aunque también puede introducir redundancia.

Por otro lado, el detector ISS selecciona puntos clave basándose en la estructura geométrica local de la nube. Para cada punto se calcula una matriz de dispersión a partir de sus vecinos y se analizan sus autovalores. Un punto se considera característico si presenta diferencias significativas entre estos valores, lo que indica que se encuentra en una zona geoméricamente distintiva.

En general, ISS suele detectar menos puntos que Harris, pero estos tienden a ser más estables y discriminativos, lo que puede mejorar la calidad de las correspondencias en el proceso de registro.

4.1.2 Filtrado de descriptores inválidos

En la implementación práctica se ha observado que algunos descriptores pueden contener valores no válidos (NaN o infinitos), especialmente en zonas con poca estructura o donde no se pueden calcular correctamente las normales.

Para evitar errores en fases posteriores, se ha implementado un filtrado que elimina:

- Descriptores inválidos
- Los keypoints asociados a dichos descriptores

Este paso es importante porque mantiene la correspondencia índice a índice entre keypoints y descriptores, garantizando la consistencia de los datos.

4.1.3 Conversión de nubes RGB a XYZ

Dado que muchos algoritmos trabajan únicamente con información geométrica, se ha añadido una conversión explícita de la nube de tipo PointXYZRGB a PointXYZ.

Este proceso elimina la información de color y conserva únicamente las coordenadas espaciales, reduciendo el coste computacional en etapas posteriores.

Detección de puntos clave

Este algoritmo analiza la geometría local de la nube y selecciona aquellos puntos que presentan cambios significativos en su entorno, como bordes, esquinas o zonas con variaciones de profundidad.

El objetivo de este paso es reducir el número de puntos de la nube original a un conjunto más pequeño de puntos representativos llamados keypoints. Estos puntos contienen la información más relevante de la estructura del entorno.

En las pruebas realizadas, una nube con aproximadamente 26.000 puntos se reduce a unos 400 keypoints.

Cálculo de normales

Para describir correctamente la geometría alrededor de cada keypoint es necesario conocer la orientación local de la superficie. Para ello se calculan las normales de la nube utilizando el algoritmo NormalEstimation.

Una normal es un vector que indica la dirección perpendicular a la superficie en cada punto. Esta información se obtiene analizando la distribución de puntos en un vecindario alrededor de cada punto de la nube.

Cálculo del FPFH

Este descriptor analiza la forma del entorno alrededor de cada keypoint y genera un vector numérico que describe la distribución de las normales en su vecindario. Este vector funciona como una especie de “firma geométrica” del punto.

Cada keypoint tiene asociado un descriptor FPFH, lo que permite comparar puntos de diferentes nubes de puntos en etapas posteriores del algoritmo.

```
Puntos nube original: 26271
Keypoints SIFT: 408
Descriptores FPFH: 408
```

DETECTOR: HarrisKeypoint3D
DESCRIPTOR: SHOTEstimation

Conversión de los puntos clave

El detector Harris devuelve los puntos clave en el formato PointXYZI, que incluye un campo adicional denominado intensity. Este campo almacena el valor de respuesta del detector Harris para cada punto.

Sin embargo, para los pasos posteriores del pipeline únicamente se necesitan las coordenadas espaciales del punto. Por este motivo se realiza una conversión de los keypoints al formato PointXYZ, conservando únicamente la información geométrica.

Esta conversión simplifica el procesamiento posterior y permite utilizar los keypoints con distintos tipos de descriptores.

Estimación de normales

Las normales describen la orientación local de la superficie alrededor de cada punto y son fundamentales para muchos descriptores geométricos.

La estimación de normales se realiza analizando un conjunto de puntos vecinos dentro de un radio determinado. A partir de estos puntos se calcula el plano que mejor aproxima la superficie local, obteniendo así la dirección normal de dicha superficie. Estas normales se utilizarán para construir el descriptor SHOT.

Descriptor SHOT

Para calcular el descriptor, primero se define un vecindario alrededor del keypoint utilizando un radio de búsqueda. Dentro de este vecindario se analizan los puntos cercanos y la orientación de sus normales, que indican cómo está orientada la superficie en cada punto.

El espacio alrededor del keypoint se divide en varias regiones y, en cada una de ellas, se calculan histogramas que representan la distribución de las orientaciones de las normales de los puntos vecinos. Estos histogramas describen la forma de la superficie alrededor del keypoint.

Todos los histogramas se combinan para formar un vector descriptor final. En esta implementación se utiliza SHOT352, que genera un vector de 352 valores para cada punto clave.

Este descriptor permite describir la forma de la superficie alrededor de cada punto clave utilizando un conjunto de valores numéricos. Gracias a esto, es posible comparar puntos de diferentes nubes de puntos y comprobar si representan una zona similar de la escena.

```
Puntos nube original: 26271  
Keypoints HARRIS: 36  
Descriptores SHOT: 36
```

DETECTOR: HarrisKeypoint3D
DESCRIPTOR: FPFHEstimation

```
Puntos nube original: 26271  
Keypoints HARRIS: 36  
Descriptores FPFH: 36
```

4.2 Encontrar emparejamientos

El siguiente paso consiste en encontrar correspondencias entre la nube actual y la nube anterior. Para ello se comparan los descriptores de ambas nubes utilizando la clase `CorrespondenceEstimation`.

Cada descriptor de la nube actual se compara con los de la nube anterior para encontrar el más similar. En la implementación dos puntos se consideran emparejados únicamente si ambos son el mejor candidato del otro. Esto permite reducir emparejamientos incorrectos.

Además, en la implementación se utilizan correspondencias recíprocas. Esto significa que dos puntos se consideran correspondencia únicamente si ambos son el mejor vecino del otro. Esta estrategia reduce significativamente el número de emparejamientos erróneos y mejora la robustez del sistema.

4.3 Algoritmo RANSAC

El resultado del emparejamiento es un conjunto de correspondencias entre keypoints de ambas nubes. Sin embargo, estas correspondencias todavía pueden contener errores, por lo que se aplica un RANSAC para eliminar emparejamientos incorrectos.

RANSAC selecciona aleatoriamente subconjuntos de correspondencias, calcula una transformación candidata y evalúa cuántas correspondencias cumplen dicha transformación dentro de un umbral de error. Las correspondencias que cumplen esta condición se llaman *inliers*.

En la implementación se ha configurado un umbral de distancia para determinar si una correspondencia es válida (*inlier*), así como un número máximo de iteraciones. Además, se requiere un número mínimo de correspondencias válidas para aceptar la transformación estimada.

Si no se alcanza este mínimo, la transformación se descarta, evitando introducir errores en el mapa global.

4.4 Construir el mapa

Una vez estimada la transformación entre nubes consecutivas, la nube actual se transforma al sistema de referencia global y se añade al mapa acumulado. De esta forma, el mapa se va construyendo progresivamente a medida que el robot obtiene nuevas observaciones del entorno.

Sin embargo, cada captura de la cámara contiene una gran cantidad de puntos, por lo que al acumular varias nubes el número total puede crecer rápidamente. Para evitar este problema se utiliza el filtro `VoxelGrid` de la librería PCL, que reduce la densidad de la nube agrupando los puntos dentro de pequeños voxels y sustituyéndolos por un punto representativo.

Este proceso permite mantener un mapa global más compacto sin perder la estructura general del entorno.

4.5 Uso de ICP

El algoritmo ICP se emplea como una etapa de refinamiento en el proceso de registro de nubes de puntos. Su objetivo principal es mejorar la alineación entre dos capturas consecutivas del sensor una vez que ya se ha obtenido una estimación inicial de la transformación.

Previamente, el sistema calcula correspondencias entre puntos característicos mediante descriptores y utiliza RANSAC para estimar una transformación entre las dos nubes. Esta transformación es robusta frente a correspondencias erróneas, pero su precisión es limitada, ya que se basa únicamente en los keypoints.

ICP se utiliza para refinar la transformación inicial. A partir de esta estimación previa, el algoritmo considera ahora las nubes completas y realiza un ajuste iterativo. En cada iteración, busca para cada punto de una nube el punto más cercano en la otra y calcula la transformación que minimiza la distancia entre todos esos pares de puntos. Este proceso se repite hasta que se alcanza la convergencia o se cumple un criterio de parada.

Gracias a este refinamiento, se obtiene una alineación mucho más precisa entre las nubes, lo que resulta fundamental en aplicaciones de mapeado. En particular, permite reducir los errores acumulados al integrar múltiples capturas en un mapa global, mejorando la coherencia y calidad final del mismo.

En la implementación se establecen varios criterios de convergencia, como el número máximo de iteraciones, la distancia máxima entre correspondencias y umbrales de error. Si ICP no converge o el error final es demasiado alto, se conserva la transformación inicial obtenida por RANSAC.

Esto permite evitar empeorar la estimación en situaciones donde ICP no es fiable.

4.6 Uso de TF

Durante el desarrollo de la práctica, inicialmente se consideró el uso del sistema de transformaciones TF de ROS2 para situar las nubes de puntos en un sistema de referencia global. Sin embargo, finalmente se optó por eliminar su uso en favor de un enfoque basado únicamente en el registro mediante técnicas de visión 3D.

El motivo principal de esta decisión es que TF proporciona directamente la transformación entre distintos sistemas de referencia del robot, lo que implica que la posición de cada nube de puntos ya está resuelta mediante la propia infraestructura de ROS2. En otras palabras, TF permite conocer en todo momento la pose del robot sin necesidad de estimarla a partir de los datos sensoriales.

Sin embargo, el objetivo fundamental de esta práctica es precisamente abordar el problema del registro de nubes de puntos, es decir, estimar la transformación entre distintas capturas utilizando únicamente la información geométrica contenida en las propias nubes. Si se emplea TF, este problema queda en gran parte resuelto de antemano, ya que la alineación entre nubes se obtiene directamente a partir de las transformaciones proporcionadas por el sistema.

Por este motivo, decidimos prescindir de TF y calcular la transformación de forma incremental a partir de las correspondencias entre nubes consecutivas, utilizando algoritmos como RANSAC e ICP. En esta implementación, la transformación global se construye acumulando las transformaciones relativas estimadas entre pares de nubes, como puede observarse en el uso de una matriz de transformación global que se actualiza iterativamente en cada paso del proceso.

Este enfoque permite evaluar de manera más realista el comportamiento de los algoritmos de registro, así como analizar su precisión, robustez frente al ruido y capacidad para mantener la coherencia del mapa a lo largo del tiempo.

No obstante, esta decisión también introduce ciertas limitaciones. Al no utilizar información externa de localización, el sistema es más susceptible a la acumulación de errores (drift), especialmente en trayectorias largas. A pesar de ello, se ha considerado que este enfoque resulta más adecuado desde el punto de vista didáctico, ya que permite estudiar en profundidad el problema del registro de nubes y el impacto de los distintos algoritmos empleados.

4.7 Arquitectura del nodo ROS2

El sistema se ha implementado como un nodo de ROS2 que procesa nubes de puntos en tiempo real.

El nodo realiza las siguientes operaciones:

1. Suscripción al tópico de la cámara de profundidad

2. Filtrado de la nube de puntos (VoxelGrid, PassThrough y eliminación de outliers)
3. Transformación al sistema global mediante TF
4. Acumulación de la nube en un mapa global
5. Ejecución del pipeline de registro seleccionado
6. Publicación periódica del mapa global

4.7.1 Selección dinámica del pipeline

El sistema permite seleccionar el pipeline mediante un parámetro de ROS2. Esto facilita la experimentación sin necesidad de modificar el código.

4.7.2 Reducción de frecuencia

Para reducir la carga computacional, el nodo no procesa todas las nubes recibidas, sino una de cada N. Esto permite mantener el sistema en tiempo real sin saturar la CPU.

4.8 Comparación

Después de hacer varias pruebas con distintos descriptores y detectores hemos podido observar que el ISS + FPFH funciona mejor que el Harris + SHOT debido a:

- **Número de keypoints.** ISS ha permitido observar que este genera menos keypoints pero más robustos, lo que puede mejorar la calidad de las correspondencias en escenas con buena estructura geométrica, mientras que Harris 3D tiende a detectar un mayor número de puntos. Esto hace que con Harris haya que procesar más características.
- **Coste del descriptor.** El descriptor SHOT tiene 352 dimensiones, lo que hace que su cálculo y comparación sean más costosos. En cambio, FPFH tiene solo 33 dimensiones, por lo que es más rápido de calcular y comparar.
- **Búsqueda de correspondencias.** Al trabajar con menos keypoints y descriptores más pequeños, el proceso de encontrar correspondencias entre nubes suele ser más rápido con ISS + FPFH.
- **Eficiencia general.** Las combinaciones con FPFH siguen siendo más eficientes computacionalmente, mientras que SHOT proporciona descriptores más ricos pero con mayor coste.

La incorporación del detector ISS ha permitido observar que este genera menos keypoints pero más robustos, lo que puede mejorar la calidad de las correspondencias en escenas con buena estructura geométrica.

Por otro lado, las combinaciones con FPFH siguen siendo más eficientes computacionalmente, mientras que SHOT proporciona descriptores más ricos pero con mayor coste.

4.9 Problemas encontrados durante la práctica

Un problema especialmente relevante durante la práctica fue el relacionado con el registro de nubes de puntos y el ajuste de los parámetros de los algoritmos empleados.

El pipeline implementado depende de la correcta elección de múltiples parámetros en cada etapa. En primer lugar, el detector de keypoints requiere ajustar adecuadamente el radio de

búsqueda y el umbral de respuesta. Si estos valores no son adecuados, pueden aparecer dos situaciones problemáticas: o bien se detectan muy pocos puntos característicos, lo que dificulta encontrar correspondencias entre nubes, o bien se detectan demasiados puntos irrelevantes, introduciendo ruido en el proceso.

En la estimación de normales y el cálculo de descriptores ocurre algo similar. El radio de búsqueda define el entorno local utilizado para describir cada punto. Si es demasiado pequeño, los descriptores no capturan suficiente información geométrica y resultan poco discriminativos; si es demasiado grande, se pierde detalle local y diferentes zonas de la nube pueden volverse indistinguibles. Esto afecta directamente a la calidad de las correspondencias encontradas entre nubes consecutivas.

Como consecuencia, la fase de emparejamiento puede generar un número reducido de correspondencias válidas o incluir muchas correspondencias erróneas. Esto impacta en el algoritmo RANSAC, que necesita un mínimo de correspondencias correctas para estimar una transformación fiable. Si hay demasiados outliers o muy pocos inliers, la transformación calculada puede ser incorrecta o incluso no estimarse.

Finalmente, aunque ICP se encarga de refinar la alineación, su rendimiento también depende de la calidad de la transformación inicial. Si RANSAC proporciona una estimación demasiado alejada de la solución real, ICP puede no converger o hacerlo hacia una solución incorrecta.

5 Conclusiones

A lo largo de esta práctica se ha podido comprobar cómo la elección del detector y descriptor condiciona de forma significativa tanto la calidad del registro como el coste computacional del sistema.

En cuanto a los detectores, Harris 3D genera un mayor número de keypoints, lo que en principio ofrece más candidatos para el emparejamiento, pero a costa de introducir más ruido y aumentar el tiempo de procesamiento. ISS, por el contrario, produce un conjunto más reducido de puntos clave, pero geométricamente más estables y discriminativos, lo que se traduce en correspondencias de mejor calidad cuando la escena tiene estructura geométrica suficiente.

Respecto a los descriptores, SHOT (352 dimensiones) ofrece una descripción más rica del entorno local de cada punto, pero su coste de cálculo y comparación es considerablemente mayor. FPFH, con sus 33 dimensiones, resulta mucho más eficiente tanto en tiempo de cómputo como en la búsqueda de correspondencias, siendo la opción más adecuada para sistemas que operan en tiempo real. En consecuencia, la combinación ISS + FPFH se presenta como la más equilibrada: keypoints robustos y descriptores ligeros, lo que permite mantener una buena calidad de registro sin comprometer el rendimiento.

Otro aspecto crítico ha sido el ajuste de parámetros a lo largo del pipeline. Radios de búsqueda mal configurados en la detección de keypoints o en el cálculo de normales pueden degradar drásticamente los resultados, generando correspondencias erróneas que RANSAC no siempre logra filtrar correctamente. A su vez, si la transformación inicial proporcionada por RANSAC es demasiado imprecisa, ICP puede no converger o hacerlo hacia una solución incorrecta, lo que ilustra la fuerte dependencia entre etapas del sistema.

Como posible línea de mejora, sería interesante incorporar un mecanismo de ajuste adaptativo de parámetros en función de las métricas que el propio pipeline produce. Por ejemplo, si el número de keypoints detectados es demasiado bajo o el número de inliers obtenido por RANSAC cae por debajo de un umbral, el sistema podría reajustar automáticamente parámetros como el radio de búsqueda del detector o el umbral de distancia de RANSAC, evitando así tener que fijarlos manualmente por prueba y error y mejorando la robustez del sistema ante distintas condiciones del entorno.

References

- [1] Point Cloud Library (PCL). Official Documentation. <https://pointclouds.org/documentation/>.
- [2] Point Cloud Library. How to use Random Sample Consensus model. https://pcl.readthedocs.io/projects/tutorials/en/latest/random_sample_consensus.html.
- [3] Point Cloud Library. Downsampling a PointCloud using a VoxelGrid filter. https://pcl.readthedocs.io/projects/tutorials/en/latest/voxel_grid.html.
- [4] Point Cloud Library. Keypoints. https://pcl.readthedocs.io/projects/tutorials/en/latest/correspondence_grouping.html.
- [5] Point Cloud Library. HarrisKeypoint3D. https://pointclouds.org/documentation/classpcl_1_1_harris_keypoint3_d.html.
- [6] Point Cloud Library. Estimating Surface Normals in a PointCloud. https://pcl.readthedocs.io/projects/tutorials/en/latest/normal_estimation.html.
- [7] Point Cloud Library. Fast Point Feature Histograms (FPFH) descriptors. https://pcl.readthedocs.io/projects/tutorials/en/latest/fpfh_estimation.html.
- [8] Point Cloud Library. ISSKeypoint3D. https://pointclouds.org/documentation/classpcl_1_1_i_s_s_keypoint3_d.html.
- [9] Point Cloud Library. 3D Object Recognition based on Correspondence Grouping. https://pcl.readthedocs.io/projects/tutorials/en/latest/correspondence_grouping.html.
- [10] Point Cloud Library. How to incrementally register pairs of clouds. https://pcl.readthedocs.io/projects/tutorials/en/latest/pairwise_incremental_registration.html.
- [11] Point Cloud Library. Aligning object templates to a point cloud. https://pcl.readthedocs.io/projects/tutorials/en/latest/template_alignment.html.
- [12] Point Cloud Library. Normal Estimation Using Integral Images. https://pcl.readthedocs.io/projects/tutorials/en/latest/normal_estimation_using_integral_images.html.
- [13] Point Cloud Library. PCL Walkthrough. <https://pcl.readthedocs.io/projects/tutorials/en/latest/walkthrough.html>.